AN EMPIRICAL STUDY ON SAT ENCODING TECHNIQUES AND MULTI-PARADIGM COMPARISON (SAT, CP, MIP) FOR THE N-QUEENS PROBLEM

Tran Nhat Hung

Faculty of Information Technology, VNU University of Engineering and Technology, 144 Xuan Thuy, Cau Giay, Hanoi, Vietnam

ARTICLE INFO

Keywords:
N-Queens
SAT Solving
At-Most-One Encoding
Constraint Programming
Mixed Integer Programming
CDCL
PySAT

Abstract

The N-Queens problem is a classic benchmark in combinatorial optimization, automated reasoning, and formal verification. When the chessboard scale expands to large dimensions ($N \geq 500$, corresponding to millions of Boolean variables), the exponential explosion of the search space requires modeling methods and exact solvers to deliver exceptional performance. In this study, we implement two deep empirical research axes: (1) Investigating and benchmarking five At-Most-One (AMO) constraint encoding techniques into SAT, including Binomial (Pairwise), Binary (Bitwise), Sequential Counter, Commander Variable ($g = 3$), and 2-Product on the Glucose4 solver; (2) Extending a cross-paradigm benchmark between SAT Solving and two baseline exact computing paradigms: Constraint Programming (CP: Google OR-Tools CP-SAT, IBM ILOG CPLEX CP Optimizer) and 0-1 Mixed Integer Programming (MIP: Gurobi Optimizer, IBM ILOG CPLEX MIP) across a synchronized benchmark suite of 15 board sizes from $N = 4$ to $N = 1000$. Experimental results demonstrate that the Binomial encoding explodes with $O(N^4)$ clauses, causing Out-Of-Memory (OOM) failures at $N \geq 512$. Conversely, although the Sequential Counter technique preserves Generalized Arc Consistency (GAC), it suffers from the *Sequential Counter Paradox*, where millions of auxiliary variables dilute the CDCL solver's VSIDS branching heuristic, leading to timeouts. The Commander Variable and Binary encodings achieve an ideal balance, successfully solving all 15 benchmark instances up to $N = 1000$ (with Commander solving $N = 1000$ in only 20.10 seconds). In the cross-paradigm comparison, IBM CPLEX CP Optimizer exhibits absolute dominance (0.250 seconds for $N = 1000$) due to compact N-integer-variable modeling and Régin's filtering propagation for the global AllDifferent constraint. Notably, PySAT Glucose4 Commander completes $N = 1000$ in 18.92 seconds, outperforming both major commercial solvers, Gurobi MIP (Timeout) and OR-Tools CP-SAT (Timeout). All source code, empirical datasets, and measurement results are publicly available at GitHub: https://github.com/NhHung07/N_Queens.git.

1. Introduction

1.1. Problem Definition

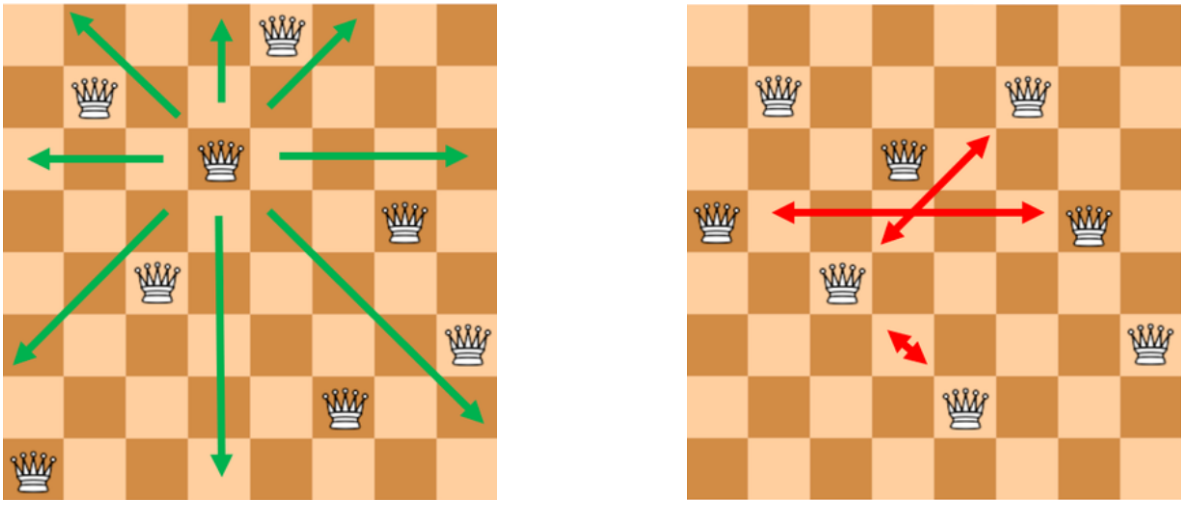
The N-Queens problem is a classic benchmark in artificial intelligence, combinatorial optimization, and automated reasoning [3]. Originally formulated by chess player Max Bezzel in 1848 for an 8×8 chessboard [1] and generalized by Franz Nauck to an $N \times N$ board in 1850 [2], the objective is to place N non-attacking queens on an $N \times N$ chessboard such that no two queens can threaten each other.

Figure 1 illustrates the visual distinction between a valid configuration and a conflicting configuration:

- **Figure 1a (Valid solution):** Every queen occupies an independent position, sharing no common row, column, or diagonal with any other queen.
- **Figure 1b (Conflicting configuration):** There exist multiple queens situated on the same row, column, or diagonal, directly attacking each other.

Course Project Report 1 – Course: Modern Topics in Computer Science (Course ID: INT3011E 3). Instructor: Dr. To Van Khanh. Student: Tran Nhat Hung – Student ID: 24021507.

24021507@vnu.edu.vn (T.N. Hung)



(a) Valid solution configuration

(b) Constraint violation configuration

Figure 1: Visual illustration of a valid solution configuration versus a conflicting configuration on the N-Queens chessboard.

1.2. Rules of the N-Queens Problem

Under standard international chess rules, the queen is the most versatile and powerful piece on the board: it can move an arbitrary number of unobstructed squares horizontally (along rows), vertically (along columns), and diagonally (along both main diagonals and anti-diagonals). Consequently, placing N queens on an $N \times N$ chessboard must simultaneously satisfy four prerequisite conditions:

1. No two queens share the same row (Row constraint).
2. No two queens share the same column (Column constraint).
3. No two queens share the same main diagonal (Main diagonal constraint).
4. No two queens share the same anti-diagonal (Anti-diagonal constraint).

Since the chessboard contains exactly N rows and N columns, and exactly N queens must be placed, by Dirichlet's pigeonhole principle, each row and each column must contain **exactly one queen** (Exactly-One). Meanwhile, each diagonal can contain **at most one queen** (At-Most-One).

1.3. Practical Applications of the Problem

Far beyond a recreational mathematical puzzle, mathematical models derived from the N-Queens problem are intimately connected to and widely applied across numerous modern engineering domains [10, 11]:

- **VLSI (Very Large Scale Integration) Circuit Design and Verification:** Optimizing routing paths and non-conflicting component placement across semiconductor integrated circuit layers.
- **Scheduling and Computational Resource Management:** Register allocation in optimizing compilers and deadlock prevention in multithreaded process scheduling.
- **Telecommunications and Optical Routing:** Wavelength assignment in wavelength-division multiplexing (WDM) optical networks ensuring parallel channels do not produce cross-talk interference.
- **Aviation and Air Traffic Management:** Runway slot allocation and landing flight scheduling at major international hubs.
- **Permutation Problems and Traveling Salesperson (TSP):** Formulating combinatorial permutation constraints in logistics routing optimization [12].

1.4. Traditional Solving Approaches

Early algorithmic approaches for solving the N-Queens problem relied primarily on exhaustive tree search combined with pruning techniques:

1. Backtracking Algorithm: The algorithm traverses sequentially through each row (or column), places a queen at the first available conflict-free square, and recursively proceeds to the next row [13]. If a dead-end state is encountered where no safe square remains, the algorithm backtracks to the previous row to try alternative positions. The procedure is formally detailed in Algorithm 1.

Algorithm 1 Solving the N-Queens Problem using Backtracking

```
function solveNQueens(N)
    Initialize array board of size N with elements set to −1
    solveNQueensUtil(board, 0, N)
end function

function solveNQueensUtil(board, row, N)
    if row = N then
        Print solution configuration printBoard(board)
        return True
    end if
    for col = 0 to N − 1 do
        if isSafe(board, row, col, N) then
            board[row] ← col                          ▷ Place queen at safe position
            if solveNQueensUtil(board, row + 1, N) then
                return True
            end if
            board[row] ← −1                           ▷ Backtrack: Remove queen
        end if
    end for
    return False
end function

function isSafe(board, row, col, N)
    for i = 0 to row − 1 do
        if board[i] = col then return False           ▷ Column conflict
        end if
        if board[i] − i = col − row then return False ▷ Main diagonal conflict
        end if
        if board[i] + i = col + row then return False ▷ Anti-diagonal conflict
        end if
    end for
    return True
end function
```

2. Branch and Bound Algorithm: To overcome the $O(N)$ safety verification overhead in the isSafe routine of classic backtracking, the Branch and Bound technique introduces three $O(1)$ state lookup tables: rowLookup of size N, slashLookup of size $2N-1$ for main diagonals, and backslashLookup of size $2N-1$ for anti-diagonals [14]. The algorithm is formally presented in Algorithm 2.

Although Branch and Bound substantially accelerates step-level checks, the overall combinatorial search space still scales factorially ($O(N!)$), rendering these traditional tree-traversal algorithms incapable of solving instances beyond $N > 30$.

Algorithm 2 Solving the N-Queens Problem using Branch and Bound

```
function solveNQueensBB(N)
    Initialize array rowLookup of size N with value False
    Initialize slashLookup, backslashLookup of size 2N − 1 with value False
    Initialize board matrix board[N][N] ← 0
    hasSol ← solveBBUtil(board, 0, rowLookup, slashLookup, backslashLookup, N)
    if not hasSol then
        print "No solution configuration found"
        return False
    end if
    Print solution configuration printBoard(board)
    return True
end function

function solveBBUtil(board, col, rowL, slashL, bslashL, N)
    if col ≥ N then
        return True
    end if
    for row = 0 to N − 1 do
        s ← row + col
        bs ← row − col + (N − 1)
        if not (rowL[row] ∨ slashL[s] ∨ bslashL[bs]) then
            board[row][col] ← 1
            rowL[row] ← True; slashL[s] ← True; bslashL[bs] ← True
            if solveBBUtil(board, col + 1, rowL, slashL, bslashL, N) then
                return True
            end if
            board[row][col] ← 0                                   ▷ Backtrack
            rowL[row] ← False; slashL[s] ← False; bslashL[bs] ← False
        end if
    end for
    return False
end function
```

1.5. SAT-based Encoding Approach for N-Queens

To solve the N-Queens problem on modern automated reasoning engines, the problem is formulated into Conjunctive Normal Form (CNF). We define an $N \times N$ matrix of Boolean decision variables $B_{i,j}$ $(0 \leq i, j < N)$:

$$B_{i,j} = \begin{cases} 1, & \text{if a queen is placed at square } (i,j), \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

The problem constraints are decomposed into two fundamental classes of Boolean constraints:

1. **At-Least-One (ALO) constraint:** $\sum_{i=1}^{m} x_i \geq 1$, simply expressed by exactly one linear disjunctive clause: $(x_1 \vee x_2 \vee \cdots \vee x_m)$.
2. **At-Most-One (AMO) constraint:** $\sum_{i=1}^{m} x_i \leq 1$.

The requirement that each row and each column contains exactly one queen is formulated as the conjunction of ALO and AMO:

$$\text{Exactly-One}(X) \iff \text{ALO}(X) \wedge \text{AMO}(X) \tag{2}$$

While ALO requires only a single clause, AMO is the central focal point of SAT encoding research. The classical Binomial (Pairwise) encoding explicitly forbids every conflicting pair:

$$\text{AMO}_{\text{Binomial}}(X) = \bigwedge_{j=0}^{m-2} \bigwedge_{k=j+1}^{m-1} (\neg x_j \vee \neg x_k) \tag{3}$$

This approach generates $\binom{m}{2} = O(m^2)$ clauses per constraint, which accumulates to a quartic $O(N^4)$ explosion in the total clause count across the entire chessboard, becoming the critical scalability bottleneck.

1.6. List of Abbreviations

Table 1 summarizes the specialized acronyms and abbreviations utilized throughout this paper.

Table 1
List of specialized technical acronyms and abbreviations.

Acronym	Full Description / Meaning
ALO	At-Least-One Constraint
AMO	At-Most-One Constraint
CDCL	Conflict-Driven Clause Learning
CNF	Conjunctive Normal Form
CP	Constraint Programming
CSP	Constraint Satisfaction Problem
DPLL	Davis-Putnam-Logemann-Loveland Algorithm
GAC	Generalized Arc Consistency
ILP	Integer Linear Programming
LBD	Literal Block Distance
MIP	Mixed Integer Programming
OOM	Out-Of-Memory Error
SAT	Boolean Satisfiability Problem
UNSAT	Unsatisfiable Instance
UP	Unit Propagation
VSIDS	Variable State Independent Decaying Sum
2WL	Two-Watched Literals Scheme

2. Background

2.1. Propositional Logic

Propositional logic provides the formal mathematical foundation for studying statements possessing a deterministic truth value: true (**True** or 1) or false (**False** or 0) [15]. Elementary propositional variables are composed through five standard logical connectives: Negation ($\neg$), Conjunction ($\wedge$), Disjunction ($\vee$), Implication ($\rightarrow$), and Equivalence ($\leftrightarrow$). The comprehensive truth table for these operators is presented in Table 2.

Table 2
Truth table of elementary propositional logic operations.

A	B	$\neg A$	$A \wedge B$	$A \vee B$	$A \rightarrow B$	$A \leftrightarrow B$
True	**True**	**False**	**True**	**True**	**True**	**True**
True	**False**	**False**	**False**	**True**	**False**	**False**
False	**True**	**True**	**False**	**True**	**True**	**False**
False	**False**	**True**	**False**	**False**	**True**	**True**

2.2. Conjunctive Normal Form (CNF)

Modern SAT solvers require the input formula to be formulated in Conjunctive Normal Form (CNF):

- **Boolean Variable:** A variable $x \in \{\textbf{False}, \textbf{True}\}$.
- **Literal:** A variable or its negation (x or $\neg x$).
- **Clause:** A disjunction ($\vee$) of a set of literals: $C = (l_1 \vee l_2 \vee \cdots \vee l_k)$. A clause evaluates to **True** if at least one constituent literal evaluates to **True**.
- **CNF Formula:** A conjunction ($\wedge$) of clauses: $\Phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$. The formula Φ is satisfied if and only if **all** constituent clauses are simultaneously satisfied.

2.3. The SAT Problem and Modern SAT Solvers

The Boolean Satisfiability (SAT) problem asks: Given a CNF formula Φ, does there exist a truth assignment to its variables that satisfies Φ? If such an assignment exists, the formula is SAT; otherwise, it is UNSAT.

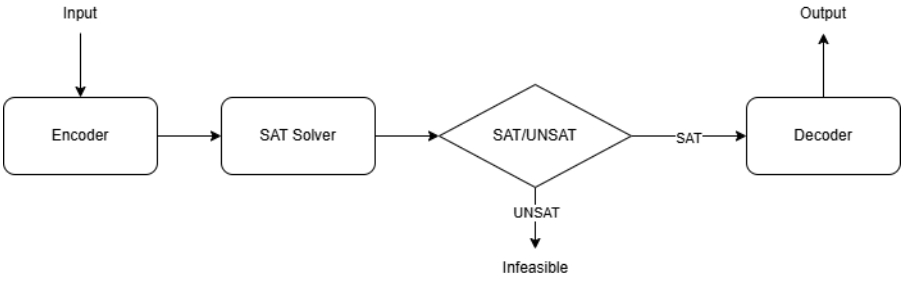


Figure 2: Overall execution pipeline of a modern SAT solver (SAT Solver Execution Flow).

As illustrated in Figure 2, the resolution pipeline starts with formalizing high-level constraints into propositional logic, encoding them into a CNF formula, passing them to the solver core, and extracting the corresponding solution assignment.

The state-of-the-art algorithmic framework implemented in modern industrial SAT solvers (e.g., Glucose4, MiniSat, Lingeling) is **Conflict-Driven Clause Learning (CDCL)** [16, 17, 18]. CDCL elevates classical DPLL search through key architectural pillars:

1. **Unit Propagation (UP) and Two-Watched Literals (2WL):** Tracking exactly two unassigned literals per clause enables constant-time $O(1)$ propagation detection without iterating over entire clause contents.
2. **Conflict Analysis and Learnt Clause Derivation:** When an assignment leads to a conflict (all literals in a clause evaluate to **False**), the solver traverses the implication graph backward to the First-UIP (Unique Implication Point) cut to derive a learnt clause, preventing the solver from re-exploring identical conflict spaces.
3. **Non-chronological Backjumping:** Directly jumping back across multiple decision levels to the highest safe assertion level following a conflict.
4. **VSIDS (Variable State Independent Decaying Sum) Branching Heuristic:** Dynamically prioritizing branching on variables with high conflict participation frequencies, attenuated by an exponential decay factor.
5. **LBD-based Clause Database Management:** Evaluating learnt clause quality using Literal Block Distance (the number of distinct decision levels in the clause); clauses with low LBD values are persistently retained in cache [17].

2.4. SAT-based Encoding Concepts

SAT encoding is the process of translating a high-level Constraint Satisfaction Problem (CSP) into an equisatisfiable CNF formula. An effective encoding must maintain an optimal balance across three dimensions:

- **CNF Clause Count:** Maintaining linear or near-linear clause counts to prevent physical RAM exhaustion.
- **Auxiliary Variable Count:** Restricting auxiliary variables to avoid inflating the Boolean decision space explored by the solver's branching heuristic.
- **Preservation of Arc Consistency (GAC):** The capability to detect and prune conflicting assignments directly via Unit Propagation without requiring deep speculative branching.

2.5. Optimized Encoding Methods for At-Most-One (AMO) Constraints

Given a set of m Boolean variables $X = \{x_1, x_2, \ldots, x_m\}$, the At-Most-One constraint enforces $\sum_{i=1}^{m} x_i \leq 1$. Four advanced encoding techniques are investigated:

2.5.1. Binary (Bitwise) Encoding

Proposed by Frisch et al. [22], this technique encodes the index of the **True** variable into binary bits:

- Requires $k = \lceil \log_2 m \rceil$ auxiliary variables $b_1, b_2, \ldots, b_k$.
- For each variable x_i, let the binary representation of $(i-1)$ be $d_1 d_2 \ldots d_k \in \{0,1\}^k$. If x_i is **True**, the auxiliary vector $(b_1, \ldots, b_k)$ must match this binary codeword:

$$x_i \to \bigwedge_{j=1}^{k} (b_j = d_j) \iff \bigwedge_{j=1}^{k} (\neg x_i \vee l_{j,d_j}) \tag{4}$$

  where $l_{j,1} = b_j$ and $l_{j,0} = \neg b_j$.
- Auxiliary variables: $O(\log m)$.
- Clauses: $m\lceil \log_2 m \rceil = O(m \log m)$.

2.5.2. Sequential Counter Encoding

Formulated by Carsten Sinz in 2005 [19], this technique mimics a hardware accumulator register counting the number of **True** assignments:

- Introduces $m-1$ auxiliary variables $s_1, s_2, \ldots, s_{m-1}$, where s_i indicates that "at least one variable in the prefix $\{x_1, \ldots, x_i\}$ is **True**".
- The clause set consists of:

$$(\neg x_1 \vee s_1) \tag{5}$$
$$(\neg s_{i-1} \vee s_i) \quad \forall i \in [2, m-1] \quad \text{(State propagation)} \tag{6}$$
$$(\neg x_i \vee s_i) \quad \forall i \in [2, m-1] \quad \text{(State assignment)} \tag{7}$$
$$(\neg s_{i-1} \vee \neg x_i) \quad \forall i \in [2, m] \quad \text{(Conflict prevention)} \tag{8}$$

- Auxiliary variables: $m - 1 = O(m)$.
- Clauses: $3m - 4 = O(m)$.
- Theoretical advantage: Strictly maintains Generalized Arc Consistency (GAC) through Unit Propagation.

2.5.3. Commander Variable Encoding

Klieber and Kwon (2007) [20] proposed partitioning the variable set X into small disjoint groups of size g (typically $g = 3$):

- For each group G_i, a "commander variable" c_i is introduced, which is **True** if and only if exactly one variable in G_i is **True**.
- Within each group G_i, local Pairwise encoding is applied ($\binom{g}{2} = 3$ clauses).
- An AMO constraint is recursively enforced across the set of commander variables $\{c_1, c_2, \ldots, c_{\lceil m/g \rceil}\}$.
- Auxiliary variables: $O(m/g)$.
- Clauses: Linear $O(m)$.

2.5.4. 2-Product Matrix Encoding

Jingchao Chen (2010) [21] proposed mapping the set of m variables onto a 2D grid of size $p \times q$ such that $p \times q \geq m$ (with $p \approx q \approx \lceil \sqrt{m} \rceil$):

- Each variable x_i is assigned a unique grid coordinate (u, v) with $1 \leq u \leq p$ and $1 \leq v \leq q$.
- Introduces two sets of auxiliary variables: row variables $r_1, \ldots, r_p$ and column variables $c_1, \ldots, c_q$.
- If x_i is **True**, both row r_u and column c_v must be **True**:

$$x_i \to r_u \wedge c_v \iff (\neg x_i \vee r_u) \wedge (\neg x_i \vee c_v) \tag{9}$$

- Enforces AMO on the row set $\{r_1, \ldots, r_p\}$ and column set $\{c_1, \ldots, c_q\}$.
- Auxiliary variables: $p + q = O(\sqrt{m})$.
- Clauses: $2m + O(p^2 + q^2) = O(m)$.

3. Proposed Methodology & Multi-Paradigm Formulation

3.1. Overview of the Methodology

To comprehensively evaluate exact solving methodologies on the N-Queens problem, two parallel research axes are deployed:

1. **Axis 1 (SAT Solving):** Implementing and benchmarking five AMO encoding techniques: Binomial (Pairwise), Binary, Sequential Counter, Commander Variable ($g = 3$), and 2-Product on an identical industrial SAT solver (PySAT Glucose4).
2. **Axis 2 (Cross-Paradigm Comparison):** Directly pitting the top-performing SAT configuration against two baseline exact paradigms: Constraint Programming (Google OR-Tools CP-SAT, IBM ILOG CPLEX CP Optimizer) and 0-1 Mixed Integer Programming (Gurobi MIP, IBM ILOG CPLEX MIP).

3.2. Formulation of SAT Encodings for N-Queens

The $N \times N$ chessboard is modeled with binary decision variables $B_{i,j}$ ($0 \leq i, j < N$). The mathematical constraints are established as follows:

Row and Column Constraints: Each row i and each column j must contain exactly one queen:

$$\text{ALO: } \bigvee_{j=0}^{N-1} B_{i,j} \quad \forall i \in [0, N-1], \qquad \bigvee_{i=0}^{N-1} B_{i,j} \quad \forall j \in [0, N-1] \tag{10}$$

$$\text{AMO: } \text{AMO}(B_{i,0}, \ldots, B_{i,N-1}) \quad \forall i, \qquad \text{AMO}(B_{0,j}, \ldots, B_{N-1,j}) \quad \forall j \tag{11}$$

Diagonal Constraints: The chessboard comprises $2N-1$ main diagonals (index $k = i-j \in [-(N-1), N-1]$) and $2N-1$ anti-diagonals (index $k = i+j \in [0, 2N-2]$). Because the single-square corner diagonals cannot contain conflicts, AMO constraints are applied strictly to diagonals with length $m \geq 2$:

$$\text{Main diagonals: AMO}(\{B_{i,j} \mid i-j=k\}), \quad \forall k \in [-(N-2), N-2] \tag{12}$$

$$\text{Anti-diagonals: AMO}(\{B_{i,j} \mid i+j=k\}), \quad \forall k \in [1, 2N-3] \tag{13}$$

3.3. Theoretical Complexity Comparison among 5 SAT Encodings

Table 3 summarizes the asymptotic complexity of the five encoding techniques for an AMO constraint of size m, as well as across the complete N-Queens problem instance.

Table 3
Theoretical complexity comparison of five SAT encoding methods on the N-Queens problem.

Encoding Technique	Aux Vars / AMO	Clauses / AMO	Total Clauses	Preserves GAC
Binomial (Pairwise)	0	$O(m^2)$	$O(N^4)$	Yes
Sequential Counter	$O(m)$	$O(m)$	$O(N^2)$	Yes
Binary (Bitwise)	$O(\log m)$	$O(m \log m)$	$O(N^2 \log N)$	No
Commander Variable	$O(m/g)$	$O(m)$	$O(N^2)$	No
2-Product	$O(\sqrt{m})$	$O(m)$	$O(N^2)$	No

3.4. Multi-Paradigm Formulations (CP and MIP)

3.4.1. Constraint Programming (CP) Model

Constraint Programming provides the most compact and natural representation using N integer decision variables $Q_0, Q_1, \ldots, Q_{N-1}$ with domain $D(Q_i) = \{0, 1, \ldots, N-1\}$, where Q_i denotes the column coordinate of the queen placed on row i:

$$\text{AllDifferent}(Q_0, Q_1, \ldots, Q_{N-1}) \quad \text{(Column conflict prevention)} \tag{14}$$

$$\text{AllDifferent}(Q_0+0, Q_1+1, \ldots, Q_{N-1}+(N-1)) \quad \text{(Anti-diagonal conflict prevention)} \tag{15}$$

$$\text{AllDifferent}(Q_0-0, Q_1-1, \ldots, Q_{N-1}-(N-1)) \quad \text{(Main diagonal conflict prevention)} \tag{16}$$

The entire formulation for an arbitrary scale N requires only N integer variables and exactly 3 global AllDifferent constraints. IBM ILOG CPLEX CP Optimizer applies Régin's propagation algorithm [23] to perform domain filtering based on maximum bipartite matching.

3.4.2. Mixed Integer Programming (MIP / 0-1 ILP) Model

Mixed Integer Programming models the problem using N^2 binary decision variables $x_{i,j} \in \{0, 1\}$ analogous to SAT:

$$\sum_{j=0}^{N-1} x_{i,j} = 1 \quad \forall i \in [0, N-1] \quad \text{(Exactly one queen per row)} \tag{17}$$

$$\sum_{i=0}^{N-1} x_{i,j} = 1 \quad \forall j \in [0, N-1] \quad \text{(Exactly one queen per column)} \tag{18}$$

$$\sum_{i-j=k} x_{i,j} \leq 1 \quad \forall k \in [-(N-2), N-2] \quad \text{(Main diagonals)} \tag{19}$$

$$\sum_{i+j=k} x_{i,j} \leq 1 \quad \forall k \in [1, 2N-3] \quad \text{(Anti-diagonals)} \tag{20}$$

The model consists of N^2 binary variables and $6N-6$ linear equality and inequality constraints, solved via Branch-and-Cut incorporating polyhedral cutting planes and continuous Simplex LP relaxation.

4. Computational Experiments and Results

4.1. Benchmark Settings and Dataset

To rigorously assess scalability, the benchmark suite is standardized across 15 problem instances:

$$N \in \{4, 8, 16, 25, 36, 49, 64, 81, 100, 125, 216, 343, 512, 729, 1000\} \tag{21}$$

The largest instance $N = 1000$ represents an ultra-scale benchmark comprising $1,000,000$ decision variables and approximately $14,000,000$ CNF clauses. A strict execution timeout of 120.0 **seconds** is enforced per run. Instances exceeding this limit are marked as `TIMEOUT`. Instances whose formula generation exceeds 16 GB RAM are designated as `SKIPPED_MEMLIMIT` to maintain system integrity.

4.2. Experimental Environment

Detailed specifications of the hardware platform and software solvers are standardized in Table 4.

Table 4
Hardware platform and software solver environment specifications.

Specification Parameter	Configuration Details
Processor (CPU)	12th Gen Intel(R) Core(TM) i5-14500H (12 cores, 8 threads, up to 4.4 GHz)
System Memory (RAM)	16.0 GB DDR4
Operating System	Ubuntu 26.04 LTS (Linux x86_64)
Execution Environment	Python 3.14.4
SAT Solver	PySAT v1.9.dev15 (Solver core: Glucose4)
CP Solver (Hybrid Engine)	Google OR-Tools v9.15.6755 (CP-SAT Solver)
CP Solver (Pure CP Engine)	IBM ILOG CPLEX Optimization Studio v22.1.1 (CP Optimizer)
MIP Solver 1	Gurobi Optimizer v13.0.3 (Academic License)
MIP Solver 2	IBM ILOG CPLEX Optimization Studio v22.1.1 (MIP Academic License)

4.3. Empirical Evaluation of the 5 SAT Encodings

Table 5 compares the execution runtimes (seconds) of the five SAT encodings on Glucose4 across all 15 benchmark sizes. Comprehensive empirical measurements encompassing variable and clause counts are compiled in Appendix A.

Table 5
Execution runtime (seconds) of five SAT encoding techniques on the Glucose4 solver across 15 problem instances ($N \in [4, 1000]$).

N	Binomial	Binary	Sequential	Commander	2-Product
4	0.00039	0.00041	0.00032	**0.00021**	0.00034
8	0.00079	0.00075	0.00077	0.00081	**0.00072**
16	0.01094	0.02093	**0.00600**	0.01760	0.10392
25	**0.00916**	0.05290	0.06090	0.17529	0.14078
36	**0.02714**	0.05251	0.16478	0.04235	7.73743
49	0.08398	**0.05116**	1.14570	0.83751	10.47424
64	0.18140	0.13639	1.65263	**0.06721**	12.07009
81	0.35869	**0.11759**	24.01935	0.27075	15.55798
100	0.68852	**0.14738**	46.52458	0.16474	15.32860
125	1.29502	**0.24989**	45.29085	0.61935	25.38504
216	6.03935	**0.81394**	89.66358	1.29397	41.69153
343	19.89517	2.31824	TIMEOUT	**2.09605**	69.55248
512	MEMLIMIT	**5.44507**	TIMEOUT	6.67240	TIMEOUT
729	MEMLIMIT	64.82834	TIMEOUT	**28.44704**	TIMEOUT
1000	MEMLIMIT	24.66207	TIMEOUT	**20.09668**	TIMEOUT

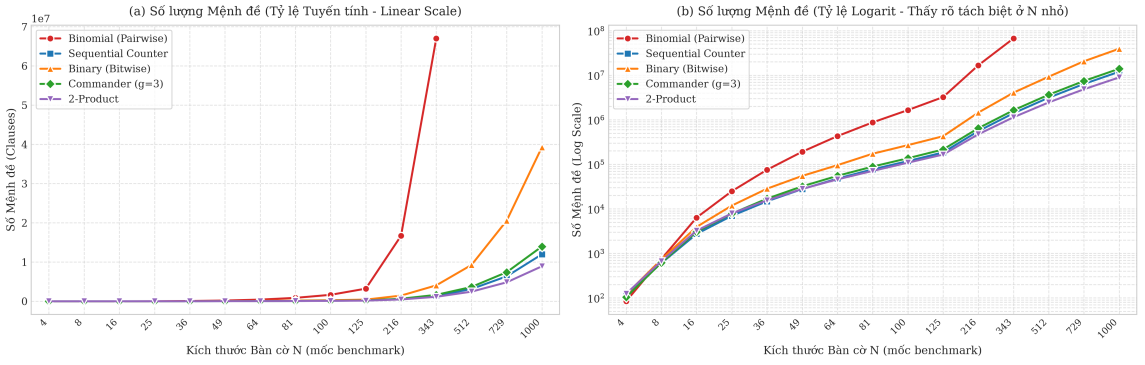


Figure 3: Comparison of generated CNF clause counts among five SAT encodings across 15 problem instances ($N \in [4, 1000]$) on linear scale (left) and logarithmic scale (right).

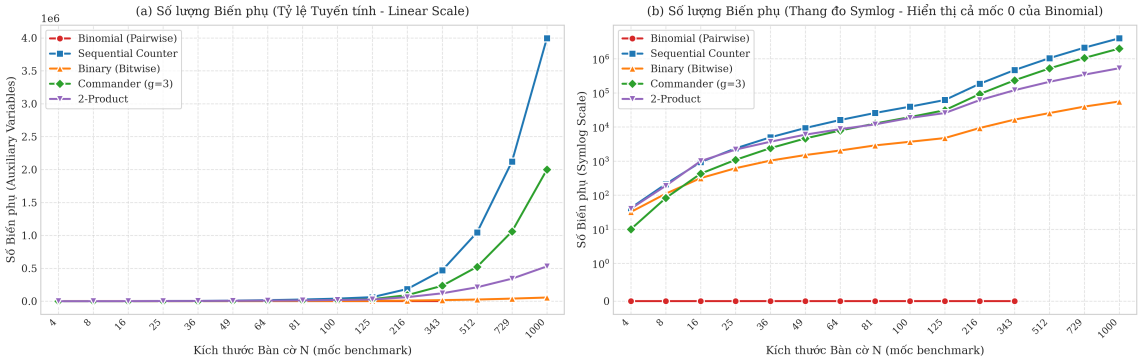


Figure 4: Comparison of auxiliary variable counts among five SAT encodings across 15 problem instances ($N \in [4, 1000]$) on linear scale (left) and symmetric log scale (right).

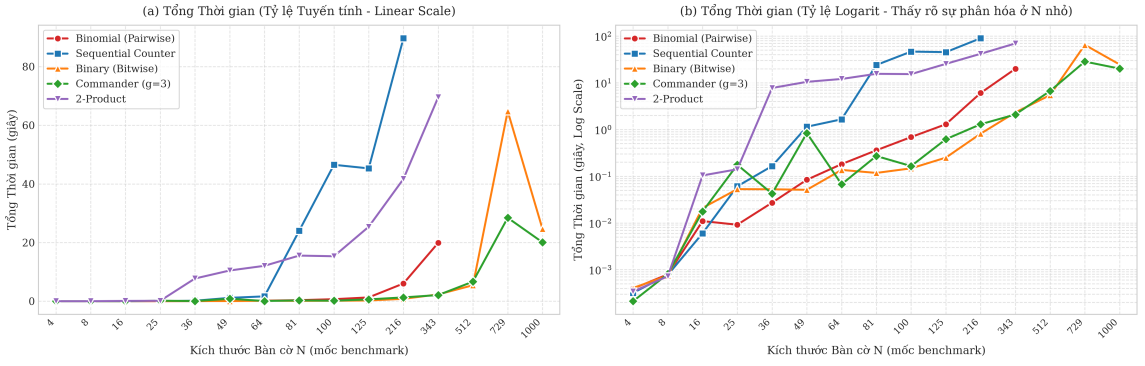


Figure 5: Comparison of total execution runtime among five SAT encodings on the Glucose4 solver across 15 problem instances ($N \in [4, 1000]$) on linear scale (left) and logarithmic scale (right).

4.4. In-Depth Empirical Analysis

1. Quartic Clause Explosion in Binomial Encoding: The Binomial encoding delivers exceptional solving speed on small matrices ($N \leq 25$) because it introduces zero auxiliary variables (0 auxiliary variables). However, the quartic growth $O(N^4)$ in clause count rapidly reveals severe limitations:

- At $N = 100$, Binomial generates over 1.64 million clauses.
- At $N = 343$, the clause count surges past 67.0 million clauses, requiring 5.09 seconds simply to construct the formula and 14.80 seconds to solve it.
- At $N \geq 512$, formula generation exceeds 16 GB of physical RAM, triggering an Out-Of-Memory error (`SKIPPED_MEMLIMIT`).

2. The Sequential Counter Paradox: Theoretically, Sequential Counter is considered an ideal encoding because it maintains a linear clause complexity $O(m)$ while strictly preserving Generalized Arc Consistency (GAC) through Unit Propagation [19]. However, empirical evidence reveals that it performs worst at scale, suffering a `TIMEOUT` starting from $N = 343$. The fundamental cause of this paradox lies in the severe disruption caused by auxiliary variables to the CDCL solver's branching mechanisms:

1. At $N = 1000$, Sequential Counter introduces $3,994,002$ auxiliary variables—nearly 4 times the number of original decision variables ($1,000,000$).
2. The state propagation clauses ($\neg s_{i-1} \vee s_i$) heavily populate the solver's unit propagation chains and implication graph. The VSIDS heuristic continually boosts the activity scores of these internal register variables s_i, misleading the solver into selecting auxiliary variables for branching decisions.
3. Branching on an internal accumulator variable s_i provides negligible search-space pruning compared to branching directly on an original board variable $B_{i,j}$, leading to search thrashing and an explosion in the search tree depth.

3. Superiority of Binary and Commander Variable Encodings:

- **Binary (Bitwise):** Demonstrates the most parsimonious auxiliary variable footprint (only $55,888$ auxiliary variables at $N = 1000$, representing less than 5.6% of original decision variables). Preserving a compact variable space enables VSIDS to remain focused on primary board variables, achieving leading performance in the mid-range $N \in [49, 216]$ (0.81s at $N = 216$) and successfully solving all 15 instances up to $N = 1000$ in 24.66 seconds.
- **Commander Variable ($g = 3$):** Achieves the optimal trade-off between reducing clause counts and controlling auxiliary variable expansion. Commander is the fastest SAT encoding at extreme scale $N \geq 343$: completing $N = 343$ in 2.10s, $N = 729$ in 28.45s (over 2× faster than Binary's 64.83s), and solving $N = 1000$ in only 20.10 seconds.

4.5. Cross-Paradigm Benchmark (SAT vs CP vs MIP)

Table 6 presents the head-to-head performance of the best-performing SAT configuration (PySAT Glucose4 Commander) against four industrial CP and MIP solvers across the 15 benchmark instances.

Detailed Discussion on Multi-Paradigm Performance:

1. **IBM CPLEX CP Optimizer:** Leveraging a compact representation with only N integer variables and 3 global AllDifferent constraints, coupled with Régin's bipartite matching filtering algorithm, CPLEX CP maintains solve times below 0.04 seconds for all $N \leq 343$, and solves the $N = 1000$ chessboard in merely 0.250 **seconds**. It represents the unconditionally dominant technology for the N-Queens problem.
2. **IBM CPLEX MIP:** Equipped with a full academic license, CPLEX MIP successfully solves the $N = 1000$ instance ($1,000,000$ binary variables) in 19.42 **seconds**, demonstrating the power of IBM's advanced Presolve routines and polyhedral Branch-and-Cut cuts.

Table 6
Cross-paradigm runtime benchmark (seconds) across three exact solving paradigms: SAT (Glucose4 Commander), Constraint Programming CP (OR-Tools, CPLEX CP), and Mixed Integer Programming MIP (Gurobi, CPLEX MIP) across 15 problem instances ($N \in [4, 1000]$).

N	PySAT (SAT)	OR-Tools (CP)	CPLEX (CP)	Gurobi (MIP)	CPLEX (MIP)
4	**0.00035**	0.00693	0.01040	0.00144	0.02712
8	0.00092	0.00484	0.00860	**0.00075**	0.01483
16	0.01730	0.01142	0.00836	**0.00782**	0.01695
25	0.17280	0.03455	0.01455	**0.00247**	0.02269
36	0.03943	0.03944	**0.01124**	0.01598	0.03602
49	0.81583	0.07143	0.01096	**0.00850**	0.04459
64	0.06577	0.14754	0.01125	**0.01115**	0.07155
81	0.28050	0.27732	**0.01325**	0.01780	0.09460
100	0.17549	0.44541	**0.01423**	0.03066	0.12695
125	0.66580	0.79482	**0.01832**	0.04310	0.20795
216	1.27169	9.93486	**0.02089**	0.14242	0.66035
343	2.13944	79.70357	**0.03815**	0.36060	1.94325
512	6.29781	TIMEOUT	**0.07285**	0.83847	4.98140
729	28.49975	TIMEOUT	**0.12560**	24.10297	14.90481
1000	18.91856	TIMEOUT	**0.25016**	TIMEOUT	19.41587

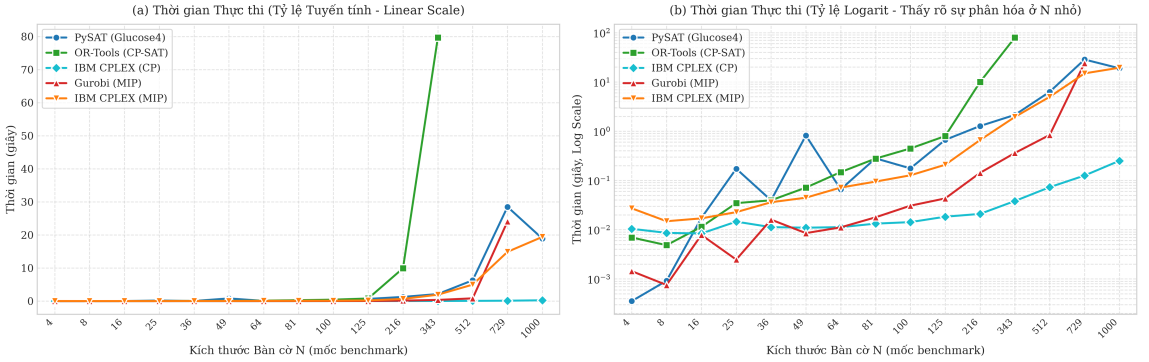


Figure 6: Multi-paradigm runtime comparison (linear scale on the left, logarithmic scale on the right) among SAT (PySAT Glucose4 Commander), Constraint Programming CP (Google OR-Tools CP-SAT, IBM CPLEX CP), and Mixed Integer Programming MIP (Gurobi MIP, IBM CPLEX MIP) across 15 problem instances ($N \in [4, 1000]$).

3. **Gurobi Optimizer (MIP):** Gurobi delivers remarkable performance up to $N = 729$: $N = 512$ $(262,144$ variables) in 0.838 seconds, and $N = 729$ $(531,441$ variables) in 24.10 seconds. However, at $N = 1000$, the expanding Branch-and-Bound search space causes the solver to hit the `TIMEOUT` limit (120s).
4. **PySAT Glucose4 (Commander Variable):** Glucose4 with Commander Variable encoding successfully completes all 15 instances up to $N = 1000$ in 18.92 **seconds** (in the solver benchmark) and 20.10 seconds (in the encoding benchmark). Remarkably, Glucose4 outperforms both Gurobi and OR-Tools CP-SAT at $N = 1000$, demonstrating that modern CDCL SAT solvers are fully capable of solving million-variable problems when equipped with optimized encoding techniques.
5. **Google OR-Tools (CP-SAT):** Demonstrates competitive performance for $N \leq 343$ (79.70 seconds), but encounters `TIMEOUT` at $N \geq 512$ due to internal Boolean flattening of global constraints, which induces search thrashing similar to the Sequential Counter encoding.

5. Conclusion

5.1. Summary of Findings

This study has presented an extensive theoretical and empirical investigation of the N-Queens problem across exact computing paradigms:

1. Thoroughly benchmarked five At-Most-One (AMO) SAT encodings: Unveiled the quartic clause explosion of Binomial encoding causing physical RAM exhaustion at $N \geq 512$, and elucidated the *Sequential Counter Paradox*, where millions of auxiliary variables dilute the CDCL solver's VSIDS branching heuristic.
2. Established the superiority of **Binary Encoding** in auxiliary variable parsimony (55.9K variables at $N = 1000$, fastest for $N \in [49, 216]$) and the balanced efficiency of **Commander Variable Encoding** (fastest for $N \geq 343$, solving $N = 1000$ in 18.92s).
3. Conducted a rigorous cross-paradigm benchmark between SAT, CP, and MIP across 15 synchronized instances up to $N = 1000$.

5.2. Technology Selection Recommendations

Based on our quantitative empirical analyses, we provide practical recommendations for technology selection:

- **For pure constraint satisfaction problems at ultra-large scale ($N \geq 500$): Constraint Programming (IBM CPLEX CP Optimizer)** is the premier solution, offering sub-second response times (0.25s for $N = 1000$).
- **For 0-1 Boolean binary formulations: PySAT Glucose4 (Commander Variable)** and **IBM CPLEX MIP** are the two standout solvers, demonstrating the capacity to solve million-variable instances within 20 seconds without memory bottlenecks.

Data availability

All algorithm source code implementations (five SAT encodings, four baseline CP and MIP models), automated benchmark execution scripts, comprehensive empirical datasets (CSV format), and visualization scripts are publicly accessible at the author's GitHub repository: `https://github.com/NhHung07/N_Queens.git`.

A. Comprehensive Detailed Experimental Data of 5 SAT Encodings

Below is the complete, detailed empirical measurement dataset of the five At-Most-One (AMO) encoding techniques evaluated on PySAT Glucose4 across 15 problem instances ($N \in [4, 1000]$).

Table 7: Complete empirical measurement data of five SAT encoding techniques on PySAT Glucose4 across 15 problem instances.

N	Technique	Total Vars	Aux Vars	Clause Count	Gen Time (s)	Solve Time (s)	Total Time (s)	Status
4	Binomial	16	0	84	0.000031	0.000361	0.000393	SAT
	Sequential	58	42	116	0.000033	0.000285	0.000318	SAT
	Binary	48	32	120	0.000065	0.000339	0.000405	SAT
	Commander	26	10	104	0.000040	0.000171	**0.000211**	SAT
	2-Product	56	40	124	0.000059	0.000281	0.000340	SAT
8	Binomial	64	0	744	0.000171	0.000622	0.000792	SAT
	Sequential	274	210	604	0.000089	0.000684	0.000773	SAT
	Binary	174	110	728	0.000144	0.000603	0.000746	SAT
	Commander	146	82	618	0.000122	0.000686	0.000808	SAT
	2-Product	252	188	668	0.000109	0.000614	**0.000723**	SAT
16	Binomial	256	0	6,352	0.000570	0.010369	0.010939	SAT
	Sequential	1,186	930	2,732	0.000194	0.005807	**0.006000**	SAT

Continued on next page ...

Table 7 – Continued from previous page

N	Technique	Total Vars	Aux Vars	Clause Count	Gen Time (s)	Solve Time (s)	Total Time (s)	Status
	Binary	572	316	3,928	0.000453	0.020475	0.020928	SAT
	Commander	682	426	3,052	0.000424	0.017172	0.017596	SAT
	2-Product	1,252	996	3,236	0.000426	0.103495	0.103921	SAT
	Binomial	625	0	24,850	0.001967	0.007194	**0.009161**	SAT
	Sequential	2,977	2,352	6,962	0.000503	0.060392	0.060895	SAT
25	Binary	1,241	616	11,806	0.001082	0.051813	0.052895	SAT
	Commander	1,709	1,084	7,810	0.001046	0.174248	0.175294	SAT
	2-Product	2,789	2,164	7,894	0.000722	0.140053	0.140775	SAT
	Binomial	1,296	0	75,252	0.006442	0.020699	**0.027140**	SAT
	Sequential	6,266	4,970	14,772	0.001122	0.163660	0.164782	SAT
36	Binary	2,328	1,032	28,320	0.002558	0.049948	0.052506	SAT
	Commander	3,690	2,394	16,904	0.002307	0.040042	0.042349	SAT
	2-Product	4,988	3,692	15,548	0.001730	7.735702	7.737431	SAT
	Binomial	2,401	0	191,394	0.030131	0.053853	0.083984	SAT
	Sequential	11,713	9,312	27,746	0.012348	1.133353	1.145701	SAT
49	Binary	3,901	1,500	54,866	0.005207	0.045954	**0.051161**	SAT
	Commander	7,009	4,608	32,014	0.003766	0.833742	0.837507	SAT
	2-Product	8,353	5,952	27,998	0.002204	10.472035	10.474239	SAT
	Binomial	4,096	0	428,864	0.068978	0.112422	0.181400	SAT
	Sequential	20,098	16,002	47,756	0.002972	1.649655	1.652627	SAT
64	Binary	6,136	2,040	95,576	0.011644	0.124750	0.136394	SAT
	Commander	11,958	7,862	55,022	0.014461	0.052746	**0.067207**	SAT
	2-Product	12,732	8,636	45,876	0.003465	12.066623	12.070088	SAT
	Binomial	6,561	0	872,802	0.149942	0.208745	0.358687	SAT
	Sequential	32,321	25,760	76,962	0.005572	24.013778	24.019350	SAT
81	Binary	9,441	2,880	172,694	0.034938	0.082653	**0.117590**	SAT
	Commander	19,177	12,616	88,778	0.013516	0.257236	0.270753	SAT
	2-Product	18,477	11,916	70,966	0.015049	15.542931	15.557980	SAT
	Binomial	10,000	0	1,646,900	0.300462	0.388057	0.688519	SAT
	Sequential	49,402	39,402	117,812	0.011384	46.513192	46.524576	SAT
100	Binary	13,678	3,678	269,024	0.051690	0.095692	**0.147383**	SAT
	Commander	29,502	19,502	136,560	0.030482	0.134258	0.164740	SAT
	2-Product	28,408	18,408	108,596	0.010247	15.318352	15.328598	SAT
	Binomial	15,625	0	3,224,250	0.509696	0.785323	1.295019	SAT
	Sequential	77,377	61,752	184,762	0.024211	45.266639	45.290850	SAT
125	Binary	20,353	4,728	426,574	0.078129	0.171765	**0.249894**	SAT
	Commander	46,685	31,060	214,918	0.041198	0.578150	0.619348	SAT
	2-Product	41,253	25,628	165,386	0.017037	25.367998	25.385035	SAT
	Binomial	46,656	0	16,703,352	2.188771	3.850577	6.039348	SAT
	Sequential	231,986	185,330	555,132	0.108629	89.554950	89.663579	SAT
216	Binary	55,988	9,332	1,449,224	0.278691	0.535247	**0.813938**	SAT
	Commander	139,002	92,346	645,382	0.161277	1.132691	1.293968	SAT
	2-Product	108,096	61,440	471,448	0.060232	41.631297	41.691528	SAT
	Binomial	117,649	0	67,021,514	5.093746	14.801421	19.895166	SAT
	Sequential	586,189	468,540	1,404,254	0.227117	-	-	TIMEOUT
343	Binary	134,109	16,460	4,060,266	0.771659	1.546584	2.318243	SAT
	Commander	351,857	234,208	1,635,926	0.304408	1.791643	**2.096051**	SAT
	2-Product	237,765	120,116	1,138,710	0.162234	69.390242	69.552476	SAT
	Binomial	262,144	0	-	-	-	-	MEMLIMIT
	Sequential	1,307,650	1,045,506	3,134,476	0.495823	-	-	TIMEOUT
512	Binary	287,730	25,586	9,262,424	1.613264	3.831807	**5.445071**	SAT
	Commander	784,502	522,358	3,652,158	0.872565	5.799838	6.672404	SAT
	2-Product	473,760	211,616	2,453,672	0.474873	-	-	TIMEOUT
	Binomial	531,441	0	-	-	-	-	MEMLIMIT
	Sequential	2,652,833	2,121,392	6,361,266	1.062963	-	-	TIMEOUT
729	Binary	571,069	39,628	20,558,002	4.206519	60.621817	64.828336	SAT
	Commander	1,590,685	1,059,244	7,413,230	1.253358	27.193685	**28.447043**	SAT
	2-Product	874,009	342,568	4,839,138	0.814480	-	-	TIMEOUT
	Binomial	1,000,000	0	-	-	-	-	MEMLIMIT
	Sequential	4,994,002	3,994,002	11,978,012	1.961011	-	-	TIMEOUT
1000	Binary	1,055,888	55,888	39,300,904	7.273818	17.388254	24.662072	SAT
	Commander	2,998,384	1,998,384	13,968,946	3.674514	16.422169	**20.096683**	SAT
	2-Product	1,529,020	529,020	8,922,032	1.757805	-	-	TIMEOUT

References

[1] M. Bezzel, Schachfreund, Berliner Schachzeitung 3 (1848) 363.

[2] F. Nauck, Briefwechsel mit allen für alle, Illustrirte Zeitung 15 (377) (1850) 182.

[3] C. Letavec, J. Ruggiero, The n-queens problem, INFORMS Transactions on Education 2 (3) (2002) 101–103.

[4] I. Rivin, I. Vardi, P. Zimmermann, The n-queens problem, The American Mathematical Monthly 101 (7) (1994) 629–639.

[5] E. M. Clarke, J. M. Wing, Formal methods: State of the art and future directions, ACM Computing Surveys 28 (4) (1996) 626–643.

[6] F. Barber, M. A. Salido, Introduction to constraint programming, Inteligencia Artificial, Revista Iberoamericana de Inteligencia Artificial 20 (2003) 13–30.

[7] S. A. Cook, The complexity of theorem-proving procedures, in: Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC'71), ACM, 1971, pp. 151–158.

[8] J. Marques-Silva, Practical applications of boolean satisfiability, in: 2008 9th International Workshop on Discrete Event Systems, IEEE, 2008, pp. 74–80.

[9] A. Biere, M. Heule, H. van Maaren, T. Walsh (Eds.), Handbook of Satisfiability, Vol. 185, IOS Press, Amsterdam, 2009.

[10] R. Sosic, J. Gu, Efficient local search with conflict minimization: A polynomial-time algorithm for the n-queens problem, IEEE Transactions on Knowledge and Data Engineering 6 (5) (1994) 661–668.

[11] C. Erbas, M. M. Tanik, Generating solutions to the n-queens problem using 2-circulant matrices, Computers & Mathematics with Applications 24 (11) (1992) 89–97.

[12] K. L. Hoffman, M. Padberg, G. Rinaldi, Traveling salesman problem, Encyclopedia of Operations Research and Management Science 1 (2013) 1573–1578.

[13] S. W. Golomb, L. D. Baumert, Backtrack programming, Journal of the ACM 12 (4) (1965) 516–524.

[14] J. R. Bitner, E. M. Reingold, Backtrack programming techniques, Communications of the ACM 18 (11) (1975) 651–656.

[15] L. Simon, Reasoning with propositional logic: From sat solvers to knowledge compilation, A Guided Tour of Artificial Intelligence Research: Volume II (2020) 115–152.

[16] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, S. Malik, Chaff: Accelerating engineering SAT runtimes, in: Proceedings of the 38th Design Automation Conference (DAC'01), ACM, 2001, pp. 530–535.

[17] G. Audemard, L. Simon, Predicting learnt clauses quality in modern SAT solvers, in: Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI'09), 2009, pp. 399–404.

[18] A. Biere, Lingeling, plingeling, picosat and precosat at sat race 2010, FMV Report Series Technical Report 10/1, Johannes Kepler University, 2010.

[19] C. Sinz, Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming (CP 2005), Vol. 3709 of LNCS, Springer, 2005, pp. 827–831.

[20] W. Klieber, G. Kwon, Efficient CNF encoding for selecting at most one item from a group, in: Proceedings of the International Workshop on Constraints in Formal Verification (CFV'07), 2007, pp. 1–9.

[21] J. Chen, A new SAT encoding of the at-most-one constraint, in: Proceedings of the 10th International Conference on Formal Methods in Computer-Aided Design (FMCAD'10), 2010, pp. 268–275.

[22] A. M. Frisch, P. A. Giannaros, SAT encodings of the at-most-k constraint, in: Proceedings of the 3rd International Workshop on Modelling and Reformulating CSPs, 2005, pp. 45–56.

[23] J.-C. Régin, A filtering algorithm for constraints of difference in CSPs, in: Proceedings of the 12th National Conference on Artificial Intelligence (AAAI'94), AAAI Press, 1994, pp. 362–367.